Query Processing and Optimization – Terminology Concepts Reference

Database Systems

Based on Lecture 9 by Tiantian Liu

Fall 2025

Contents

1 Step-by-Step Problem Solving Guide

How to Use This Chapter

This chapter teaches you **exactly how to solve exam questions** step by step. Instead of memorizing formulas, you'll learn a **systematic approach** that works for every problem type.

The Three Question Types You'll See:

1. **Join Cost Calculations** – "Calculate the I/O cost for X join"
2. **Sorting Cost Calculations** – "Calculate the cost to sort N pages"
3. **Selection Size Estimation** – "Estimate the result size of $\sigma_{\ldots}(r)$"

Golden Rule: Always start by extracting the given numbers and converting them to what you need!

1.1 Master Template: Reading the Problem

Step 0: Extract Your Numbers (DO THIS FIRST!)

Every problem gives you raw data. Your **first job** is to convert it to useful values.

What They Give You:

- Number of tuples (rows) in each relation
- Tuples per page (how many rows fit in one page)
- Memory buffer size (M or B pages)

What You Need to Calculate:

- **Number of pages** $= \lceil \text{tuples} \div \text{tuples per page} \rceil$

Example Extraction:

"r_1 has 20,000 tuples, 25 tuples fit on one page"

- Pages in $r_1 = 20,000 \div 25 = 800$ pages

Write This Down Immediately:

Relation	Tuples	Tuples/Page	Pages
r_1	20,000	25	800
r_2	45,000	30	1,500

Memory: $M = 102$ pages (or whatever is given)

1.2 Solving Join Problems

Type 1: Simple Nested-Loop Join

When asked: "Calculate I/O cost for simple nested-loop join"
The Formula:

$$\text{Cost} = (\text{tuples in outer}) \times (\text{pages in inner}) + (\text{pages in outer})$$

Step-by-Step Method:

1. Identify which relation is "outer" (the one you loop through first)
2. Calculate: (number of tuples in outer) × (pages in inner)
3. Add: pages in outer (you read outer once)

Worked Example:

r_1: 20,000 tuples, 800 pages r_2: 45,000 tuples, 1,500 pages

If r_1 is outer:

$$\text{Cost} = 20,000 \times 1,500 + 800 = 30,000,800 \text{ I/Os}$$

If r_2 is outer:

$$\text{Cost} = 45,000 \times 800 + 1,500 = 36,001,500 \text{ I/Os}$$

Key Insight: Use the relation with **fewer tuples** as outer for lower cost!

Type 2: Page-Oriented Nested-Loop Join

When asked: "Calculate I/O cost for page-oriented nested-loop join"
The Formula:

$$\text{Cost} = (\text{pages in outer}) \times (\text{pages in inner}) + (\text{pages in outer})$$

Step-by-Step Method:

1. Get number of pages for both relations
2. Multiply: (pages in outer) × (pages in inner)
3. Add: pages in outer

Worked Example:

r_1: 800 pages r_2: 1,500 pages

If r_1 is outer:

$$\text{Cost} = 800 \times 1,500 + 800 = 1,200,800 \text{ I/Os}$$

If r_2 is outer:

$$\text{Cost} = 1,500 \times 800 + 1,500 = 1,201,500 \text{ I/Os}$$

Key Insight: Use the relation with **fewer pages** as outer!

Type 3: Block Nested-Loop Join

When asked: "Calculate I/O cost for block nested-loop join with M buffers"
The Formula:

$$\text{Cost} = \left\lceil \frac{\text{pages in outer}}{M-2} \right\rceil \times (\text{pages in inner}) + (\text{pages in outer})$$

Why M-2? You need 1 page for inner relation input, 1 page for output. The rest $(M-2)$ holds the outer relation block.
Step-by-Step Method:

1. Calculate: $\lceil \text{outer pages} \div (M-2) \rceil$ = number of blocks
2. Multiply: blocks $\times$ inner pages
3. Add: outer pages

Worked Example:

r_1: 800 pages $\quad r_2$: 1,500 pages $\quad M = 102$

If r_1 is outer:

$$\text{Blocks} = \lceil 800 \div 100 \rceil = 8$$

$$\text{Cost} = 8 \times 1,500 + 800 = 12,800 \text{ I/Os}$$

If r_2 is outer:

$$\text{Blocks} = \lceil 1,500 \div 100 \rceil = 15$$

$$\text{Cost} = 15 \times 800 + 1,500 = 13,500 \text{ I/Os}$$

Key Insight: Smaller outer relation = fewer blocks = fewer inner scans!

Type 4: Sort-Merge Join

When asked: "Calculate I/O cost for sort-merge join"
First, Check the Shortcut Condition:

$$\frac{|R|}{M} + \frac{|S|}{M} < M \quad ?$$

If YES: Use the **simple formula**:

$$\text{Cost} = 3 \times (|R| + |S|)$$

If NO: Calculate full sorting cost (rarely needed in exams).
Worked Example:

r_1: 800 pages $\quad r_2$: 1,500 pages $\quad M = 102$

Check condition:

$$\frac{800}{102} + \frac{1,500}{102} = 7.8 + 14.7 = 22.5 < 102 \quad \checkmark$$

Use simple formula:

$$\text{Cost} = 3 \times (800 + 1,500) = 3 \times 2,300 = 6,900 \text{ I/Os}$$

Why 3? Read both (1), write partitions (1), read to merge (1) = 3 total passes.

Type 5: Hash Join

When asked: "Calculate I/O cost for hash join"
The Formula (Always):

$$\text{Cost} = 3 \times (|R| + |S|)$$

That's it! Hash join always costs 3 times the total pages (assuming no recursive partitioning).
Worked Example:

r_1: 800 pages r_2: 1,500 pages

$$\text{Cost} = 3 \times (800 + 1,500) = 6,900 \text{ I/Os}$$

Why 3? Partition both relations (read + write = 2), then match (read = 1). Total = 3.

1.3 Join Cost Summary Card

Quick Reference: All Join Formulas

Copy this table to your exam notes:

Join Type	**Formula (R outer, S inner)**
Simple Nested-Loop	$(\text{tuples}_R) \times \lvert S\rvert + \lvert R\rvert$
Page Nested-Loop	$\lvert R\rvert \times \lvert S\rvert + \lvert R\rvert$
Block Nested-Loop	$\lceil \lvert R\rvert/(M-2)\rceil \times \lvert S\rvert + \lvert R\rvert$
Sort-Merge	$3(\lvert R\rvert + \lvert S\rvert)$ if condition met
Hash Join	$3(\lvert R\rvert + \lvert S\rvert)$

Decision Rule: Use smaller relation as outer for nested-loop joins!

1.4 Solving Selection Size Estimation Problems

Type 1: Equality Selection ($\sigma_{A=v}$)

When asked: "Estimate the size of $\sigma_{A=10}(r)$"
The Formula:

$$\text{Result size} = \frac{n_r}{V(A, r)}$$

Where:

- n_r = number of tuples in relation r
- $V(A, r)$ = number of **distinct** values of attribute A in r

Step-by-Step:

1. Find n_r (total tuples)
2. Find $V(A, r)$ (distinct values for the attribute in the condition)

3. Divide: $n_r \div V(A, r)$

Worked Example:

$r(A, B, C)$ with $n_r = 10,000$ and $V(A, r) = 500$. Estimate $\sigma_{A=10}(r)$.

$$\text{Result} = \frac{10,000}{500} = 20 \text{ tuples}$$

Intuition: If there are 500 distinct A values and tuples are evenly distributed, each value appears in $10,000/500 = 20$ tuples.

Type 2: Range Selection ($\sigma_{A<v}$ or $\sigma_{A\leq v}$)

When asked: "Estimate the size of $\sigma_{C<10}(r)$ given range [min, max]"
The Formula:

$$\text{Result size} = n_r \times \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$$

Step-by-Step:

1. Identify the range: $[\min, \max]$
2. Calculate the fraction: $(v - \min) \div (\max - \min)$
3. Multiply by total tuples: $n_r \times \text{fraction}$

Worked Example:

r has $n_r = 10,000$ tuples. Attribute C ranges from 7 to 59 (uniformly distributed). Estimate $\sigma_{C<10}(r)$.

$$\text{Fraction} = \frac{10 - 7}{59 - 7} = \frac{3}{52}$$

$$\text{Result} = 10,000 \times \frac{3}{52} \approx 577 \text{ tuples}$$

Intuition: Values below 10 cover 3/52 of the entire range, so roughly 3/52 of tuples qualify.

1.5 Solving External Sorting Problems

External Merge Sort Cost

When asked: "Calculate the I/O cost to sort N pages with M buffer pages"
The Formula:

$$\text{Cost} = 2N \times \left(\lceil \log_{M-1} \lceil N/M \rceil \rceil + 1\right)$$

Step-by-Step:

1. Calculate initial runs: $\lceil N/M \rceil$
2. Calculate merge passes: $\lceil \log_{M-1}(\text{initial runs}) \rceil$
3. Total passes = merge passes + 1 (for initial run creation)

4. Multiply: $2N \times$ total passes

Worked Example:

Sort 1,000 pages with $M = 100$ buffer pages.

1. Initial runs: $\lceil 1000/100 \rceil = 10$
2. Merge passes: $\lceil \log_{99} 10 \rceil = 1$ (since $99^1 = 99 \geq 10$)
3. Total passes: $1 + 1 = 2$
4. Cost: $2 \times 1000 \times 2 = 4,000$ I/Os

Why 2N per pass? Every pass reads all pages AND writes all pages.

1.6 Common Exam Mistakes to Avoid

Critical Mistakes That Cost You Points

Mistake 1: Using tuples instead of pages (or vice versa)

- Simple nested-loop uses **tuples** for outer
- All other joins use **pages**
- Always double-check which one the formula needs!

Mistake 2: Forgetting the "$+|R|$" term

- You always read the outer relation once
- Don't forget to add $|R|$ at the end of nested-loop formulas

Mistake 3: Using M instead of (M-2) for block nested-loop

- Block nested-loop reserves 2 pages (input + output)
- Available for outer blocks $= M - 2$

Mistake 4: Forgetting to convert tuples to pages

- Problem says "20,000 tuples, 25 per page"
- You need: $20,000 \div 25 = 800$ pages
- Do this conversion FIRST before any calculation

Mistake 5: Wrong relation as outer

- For nested-loop joins: smaller relation should be outer
- If they ask "which is better?" – calculate BOTH and compare

1.7 The Ultimate Problem-Solving Checklist

Before You Write Anything: The 5-Point Checklist

Check these EVERY time:

- ☐ **Did I convert tuples to pages?**
  - Pages = Tuples ÷ Tuples-per-page
- ☐ **Did I identify which relation is outer/inner?**
  - Usually: smaller = outer (for nested-loop)
- ☐ **Am I using the right unit in the formula?**
  - Simple NL: tuples for outer
  - Everything else: pages
- ☐ **Did I use (M-2) for block nested-loop?**
  - Not M, not (M-1)
- ☐ **Did I add the "read outer once" term?**
  - $+|R|$ at the end of nested-loop formulas

1.8 Practice Problem Walkthrough

Complete Example: Exam-Style Problem

Problem: Relations r_1 and r_2 have:

- r_1: 20,000 tuples, 25 tuples/page
- r_2: 45,000 tuples, 30 tuples/page
- $M = 102$ buffer pages

Calculate I/O cost for: (a) Simple NL, (b) Page NL, (c) Block NL, (d) Sort-Merge, (e) Hash Join

STEP 0: Convert to pages

- $|r_1| = 20,000 \div 25 = 800$ pages
- $|r_2| = 45,000 \div 30 = 1,500$ pages
- r_1 is smaller, so use it as outer for nested-loops

(a) Simple Nested-Loop (r_1 outer):

$$\text{Cost} = 20,000 \times 1,500 + 800 = 30,000,800$$

(b) Page Nested-Loop (r_1 outer):

$$\text{Cost} = 800 \times 1,500 + 800 = 1,200,800$$

(c) Block Nested-Loop (r_1 outer):

$$\text{Blocks} = \lceil 800/100 \rceil = 8$$

$$\text{Cost} = 8 \times 1,500 + 800 = 12,800$$

(d) Sort-Merge:
Check: $800/102 + 1500/102 = 22.5 < 102$ ✓

$$\text{Cost} = 3 \times (800 + 1,500) = 6,900$$

(e) Hash Join:

$$\text{Cost} = 3 \times (800 + 1,500) = 6,900$$

Summary: Hash Join and Sort-Merge are best (6,900 I/Os)

2 Understanding Notation and Terminology

Plain English Guide - Understanding the Symbols

Throughout this cheatsheet, we use specific notation. Here's what everything means:
Common Symbols:

- N = Total number of pages in a file
- $|R|$ = Number of pages in relation R (read as "size of R")
- $|S|$ = Number of pages in relation S (read as "size of S")
- B = Number of buffer pages available in memory
- p_r = Number of tuples (rows) per page in relation R
- $\lceil x \rceil$ = Ceiling function (round UP to nearest integer)
- Example: $\lceil 5.2 \rceil = 6$, $\lceil 10.9 \rceil = 11$

What is I/O Cost?

- I/O = Input/Output operations (reading/writing pages from/to disk)
- **Plain English:** How many times we need to access the disk
- **Why it matters:** Disk access is SLOW (1000x slower than memory)
- Our goal: Minimize I/O operations to make queries faster

Understanding "Outer" and "Inner" Relations:

- **Outer relation**: The table we scan first (the "main" table)
- **Inner relation**: The table we compare against (scanned repeatedly)
- **Analogy:** Like nested loops in programming - outer loop vs inner loop
- **Key insight:** Choice matters! Use the smaller table as outer.

What is a "Pass"?

- **Pass**: One complete scan through all data
- Each pass reads AND writes all pages (cost = 2N per pass)
- **Plain English:** Think of it as one round of processing

3 Quick Reference Guide

Essential Concepts - At a Glance

External Merge Sort:

- **2-Way**: Uses 3 buffer pages, Cost = $2N \times (\lceil \log_2 N \rceil + 1)$
- **Plain English:** Merge two runs at a time; very simple but expensive

- **Multi-Way**: Uses B buffer pages, Cost = $2N \times (\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1)$
- **Plain English:** Merge many runs at once; uses more memory but much faster
- Phase 1: Create sorted runs of size B
- Phase 2: Merge (B-1) runs at a time

Join Strategies (R as outer, S as inner):

- **Simple Nested-Loop**: $|R| + (p_r \times |R|) \times |S|$
- **Plain English:** For every row in R, scan all of S - extremely slow!
- **Page-Oriented Nested-Loop**: $|R| + |R| \times |S|$
- **Plain English:** For every page in R, scan all of S - still slow
- **Block Nested-Loop**: $|R| + \lceil |R|/(B-2) \rceil \times |S|$
- **Plain English:** Load many pages of R at once - much better!
- **Sort-Merge**: Sort R + Sort S + $(|R| + |S|)$ or $3(|R| + |S|)$ if optimized
- **Plain English:** Sort both tables, then merge - good for large tables
- **Hash Join**: $3(|R| + |S|)$
- **Plain English:** Partition both tables by hash - very efficient!

Logical Query Optimization Steps:

1. Break up conjunctive selections and push down
2. **Plain English:** Apply filters as early as possible to reduce data
3. Combine selections with Cartesian products (introduce joins)
4. **Plain English:** Replace expensive cross products with joins
5. Determine join order (heuristic: selections first)
6. **Plain English:** Do cheap operations first, expensive ones last
7. Introduce and push down projections
8. **Plain English:** Only keep columns you need, discard others early

SQL Evaluation Order:

1. FROM (Cartesian product) - **Create all possible row combinations**
2. WHERE (predicates) - **Filter rows**
3. GROUP BY - **Group rows by common values**
4. HAVING - **Filter groups**
5. SELECT (projection) - **Choose which columns to output**

4 Introduction: Why Query Processing Matters

The Big Picture

What is Query Processing?

- **The Problem:** You write SQL (declarative) - you say WHAT you want
- **The Challenge:** Database must figure out HOW to get it efficiently
- **The Solution:** Query processing algorithms and optimization

Why Does This Matter?

- **Performance:** Same query can take 1 second or 1 hour!
- **Scalability:** As data grows, bad algorithms become unusable
- **Cost:** Slow queries = more servers = more money
- **User Experience:** Nobody wants to wait for slow queries

Key Insight:

- Disk I/O is the bottleneck (1000x slower than RAM)
- Our goal: Minimize disk accesses
- Strategy: Use clever algorithms + available memory wisely

Real-World Example:

- Query: Find all students who took "CS101"
- **Bad approach:** Load entire database, scan everything
- **Good approach:** Use index, read only relevant pages
- **Difference:** 1,000,000 I/Os vs 10 I/Os!

5 Sorting

Why Sorting in Databases?

The Central Question: Why can't we just use regular sorting (quicksort, mergesort)?
The Problem:

- **Real databases are HUGE:** Millions or billions of rows
- **Memory is LIMITED:** Can't load everything into RAM
- **Plain English:** Imagine sorting a library with 1 million books, but your desk can only hold 100 books at a time
- **Solution needed:** External sorting (sort using disk + limited memory)

When Do We Need Sorting?

- **Eliminating duplicates**: `SELECT DISTINCT name FROM students`

- **Plain English:** Sort all names together, then duplicates are adjacent
- **Bulk loading B+-trees**: Building indexes efficiently
- **Plain English:** Sorted data loads into trees much faster
- **Ordered output**: `SELECT * FROM students ORDER BY gpa`
- **Plain English:** Users want results in specific order
- **Join algorithms**: Sort-merge join requires sorted inputs
- **Plain English:** Sorting first makes joining faster

The Big Challenge:

- **Challenge:** Data too large to fit in memory (could be terabytes!)
- **Solution:** External sorting - use disk as "extended memory"
- **Key idea:** Sort small chunks in memory, then merge them
- **Analogy:** Like sorting a deck of cards by breaking it into small piles, sorting each pile, then merging them back together

5.1 2-Way External Merge Sort

Understanding 2-Way External Merge Sort

The Big Idea:

- **Problem:** File too big to sort in memory all at once
- **Solution:** Divide and conquer!
- **Phase 1:** Create small sorted chunks ("runs")
- **Phase 2:** Repeatedly merge pairs of runs until done
- **Analogy:** Like sorting a huge pile of papers by first sorting small stacks, then merging stacks together

Why "2-Way"?

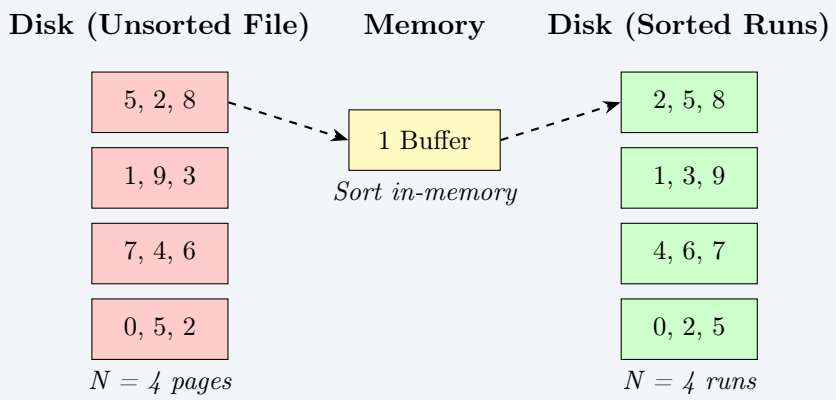
- We merge TWO runs at a time (hence "2-way")
- Simple but not optimal (we'll see better approaches later)
- Uses only 3 buffer pages: 2 input + 1 output

Visual Intuition:

- Think of a tournament bracket: pairs compete, winners advance
- Each "competition" is merging two sorted runs
- Keep merging until one winner (final sorted file)

2-Way External Merge Sort - Phase 1: Create Sorted Runs

Goal: Transform unsorted file into many small sorted pieces



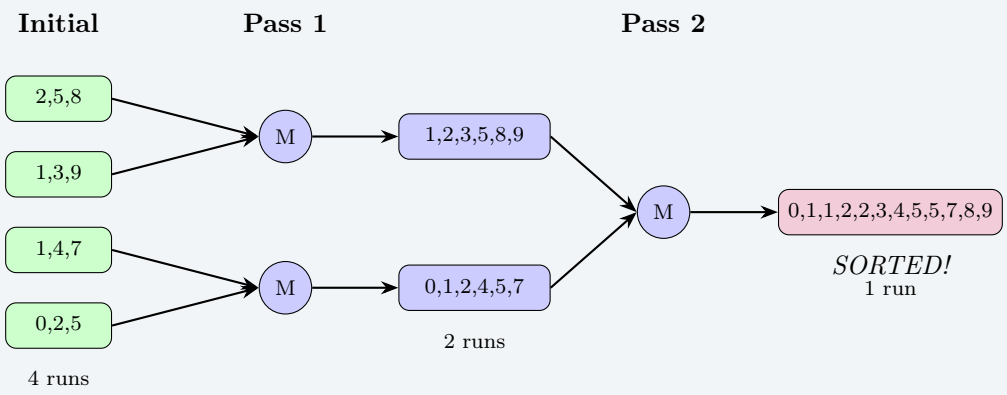
Step-by-Step Process:

1. **Read** one page from disk into memory buffer
2. **Sort** that page in memory (use quicksort, etc.)
3. **Write** sorted page back to disk as a "run"
4. **Repeat** for all N pages

Result: N pages → N sorted runs (each run = 1 page)
I/O Cost: Read N pages + Write N pages = $\boxed{\text{2N I/Os}}$

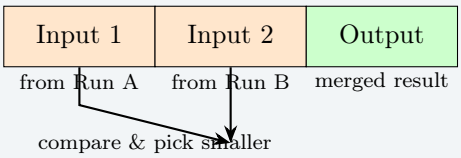
2-Way External Merge Sort - Phase 2: Merge Runs

Goal: Combine sorted runs into one final sorted file



The Merging Algorithm (How M works):

3 Buffer Pages in Memory



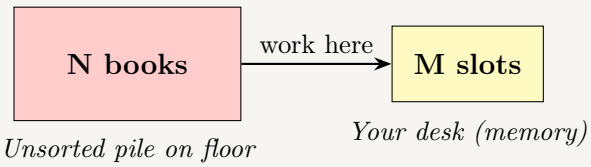
How Many Passes?

- Each pass halves the number of runs: $N \to N/2 \to N/4 \to ... \to 1$
- Number of passes $= \lceil \log_2 N \rceil$
- **Example:** N=8: Pass 1 (8→4), Pass 2 (4→2), Pass 3 (2→1) = 3 passes

I/O Cost per Pass: Read all N pages + Write all N pages = 2N I/Os
Total Phase 2 Cost: $2N \times \lceil \log_2 N \rceil$

Intuition: What Do M

The Big Picture – A Physical Analogy:
Imagine you have a **giant pile of N books** to alphabetize, but your **desk (memory) can only hold M books** at a time.



What Each Variable Represents:

Symbol	Meaning	Analogy
N	Total pages of data	Total books to sort
M	Memory buffer size	Desk size (books it holds)
$\log N$	Number of merge passes	Rounds of merging needed

Why $\log_2 N$ Passes? – The Tournament Intuition:
Think of a **knockout tournament** with N teams:

- **Round 1:** N teams → N/2 winners
- **Round 2:** N/2 teams → N/4 winners
- **Round 3:** N/4 teams → N/8 winners
- ... keep halving until 1 champion remains

How many rounds? You halve N until you reach 1:

$$N \to \frac{N}{2} \to \frac{N}{4} \to \cdots \to 1 \quad \Rightarrow \quad \log_2 N \text{ rounds}$$

Same for merge sort:

- Start with N sorted runs (1 page each)

- Each pass merges pairs: N → N/2 → N/4 → ... → 1
- **Number of passes** $= \log_2 N$

Why 2N per Pass? – The "Touch Everything Twice" Rule:
Every pass must:

1. **READ** all N pages from disk into memory
2. **WRITE** all N pages back to disk (merged)

$$\text{Cost per pass} = N \text{ (reads)} + N \text{ (writes)} = 2N$$

The Complete Formula – Putting It Together:

$$\boxed{\text{Total Cost} = \underbrace{2N}_{\text{Phase 1: create runs}} + \underbrace{2N \times \lceil \log_2 N \rceil}_{\text{Phase 2: merge passes}} = 2N \times (\lceil \log_2 N \rceil + 1)}$$

Intuitive Summary:

- **N controls the "amount of work"** – more data = more I/O per pass
- $\log_2 N$ **controls "how many times"** – more data = more rounds (but grows slowly!)
- **The +1** is for creating initial sorted runs (Phase 1)
- **The 2** is because every pass reads AND writes everything

Why Logarithm is Your Friend:

- $\log_2(1000) \approx 10$ – only 10 passes for 1000 pages!
- $\log_2(1,000,000) \approx 20$ – only 20 passes for a million pages!
- **Doubling N only adds ONE more pass** – this is why merge sort scales well

External Merge Sort – Complete Formula Reference Table

Master Table: All Formulas at a Glance

Component	2-Way Merge Sort	Multi-Way Merge Sort	Quick Calculation Trick
blue!10**PHASE 1: Create Sorted Runs**			
Run Length	1 page per run	B pages per run	*2-Way: Always 1; Multi: = buffer size*
Number of Runs	N runs	$\lceil N/B \rceil$ runs	*Divide total pages by run length*
I/O Cost	$2N$	$2N$	*Always 2N (read all + write all once)*
green!10**PHASE 2: Merge Passes**			
Merge Factor	2 (merge 2 at a time)	$B - 1$ (merge B-1 at a time)	*2-Way: always 2; Multi: buffers - 1*
Number of Passes	$\lceil \log_2 N \rceil$	$\lceil \log_{B-1} \lceil N/B \rceil \rceil$	*"How many times divide runs by merge factor?"*
Cost per Pass	$2N$	$2N$	*Always 2N (read + write everything)*
Phase 2 Total	$2N \times \lceil \log_2 N \rceil$	$2N \times \lceil \log_{B-1} \lceil N/B \rceil \rceil$	*Multiply: 2N × passes*
yellow!15**TOTAL COST**			
Formula	$2N(\lceil \log_2 N \rceil + 1)$	$2N(\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1)$	*2N × (Phase 2 passes + 1)*

Step-by-Step Calculation Guide:

Step	2-Way	Multi-Way (with B buffers)
1	Runs after Phase 1 = N	Runs after Phase 1 = $\lceil N/B \rceil$
2	Phase 2 passes = $\lceil \log_2 N \rceil$	Phase 2 passes = $\lceil \log_{B-1}(\text{runs}) \rceil$
3	Total passes = Phase 2 + 1	Total passes = Phase 2 + 1
4	Cost = $2N \times$ total passes	Cost = $2N \times$ total passes

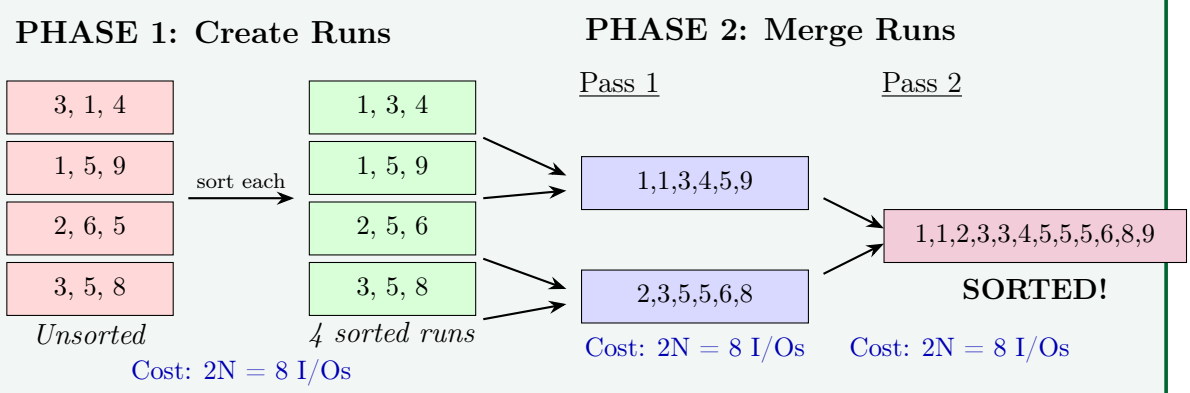
Quick Tricks for Phase 2 Pass Calculation:

- **2-Way:** "How many times can I halve N before reaching 1?"
  - $N = 8$: $8 \to 4 \to 2 \to 1 = 3$ passes ✓ ($\log_2 8 = 3$)
  - $N = 1000$: Use calculator or know $2^{10} = 1024 \approx 1000$, so ≈ 10 passes
- **Multi-Way:** "How many times can I divide runs by (B-1)?"
  - If runs $\leq B - 1$: Only 1 pass needed!
  - If $(B-1)^2 \geq$ runs $> B - 1$: 2 passes needed
- **Shortcut:** If $\lceil N/B \rceil \leq B - 1$, then Phase 2 = 1 pass (very common!)

Memory Trick for the "+1":

- The +1 is ALWAYS for Phase 1 (creating initial runs)
- Think: "One pass to create, log passes to merge"

2-Way Merge Sort - Complete Visual Example (N



Total Cost = 2N × (passes + 1) = 2(4) × (2 + 1) = 24 I/Os

Formula: $2N \times (\lceil \log_2 N \rceil + 1)$ where $\lceil \log_2 4 \rceil = 2$

2-Way External Merge Sort - Total Cost

Total I/O Cost:

$$\text{Cost} = \text{Phase 1} + \text{Phase 2} = 2N + 2N \times \lceil \log_2 N \rceil = 2N \times (\lceil \log_2 N \rceil + 1) \tag{1}$$

N (pages)	Passes ($\lceil \log_2 N \rceil$)	Total Passes (+1)	I/O Cost
4	2	3	24
8	3	4	64
100	7	8	1,600
1,000	10	11	22,000
1,000,000	20	21	42,000,000

Key Insight: Cost grows as $N \log N$ – manageable but can we do better?

5.2 Multi-Way External Merge Sort

Multi-Way External Merge Sort - Phase 1

Improvement: Use all B buffer pages
Algorithm:

1. Read B pages at a time
2. Sort B pages in main memory
3. Write out B pages as one run

Result:

- Length of each run: B pages
- Number of runs: $\lceil N/B \rceil$

I/O Cost:

- Still 2N (read all pages once, write all pages once)

Multi-Way External Merge Sort - Phase 2

Improvement: Merge (B-1) runs at a time
Algorithm:

- Use (B-1) input buffers, 1 output buffer
- **Pass 1**: Merge (B-1) runs $\rightarrow$ runs of length $B(B-1)$
- **Pass 2**: Merge (B-1) runs $\rightarrow$ runs of length $B(B-1)^2$
- **Pass P**: Merge (B-1) runs $\rightarrow$ runs of length $B(B-1)^P$

Number of Passes:

- Need to reduce $\lceil N/B \rceil$ runs to 1 run
- Each pass reduces number of runs by factor of (B-1)
- **Number of passes**: $\lceil \log_{B-1} \lceil N/B \rceil \rceil$

I/O Cost:

- Cost per pass: 2N
- Total Phase 2 Cost: $2N \times \lceil \log_{B-1} \lceil N/B \rceil \rceil$

Multi-Way External Merge Sort - Total Cost

Total I/O Cost:

$$\text{Cost} = 2N + 2N \times \lceil \log_{B-1} \lceil N/B \rceil \rceil = 2N \times (\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1) \tag{2}$$

Example: For N = 1000, B = 100

- $\lceil N/B \rceil = \lceil 1000/100 \rceil = 10$ runs after Phase 1
- $\lceil \log_{99} 10 \rceil = 1$ pass in Phase 2
- Cost $= 2 \times 1000 \times (1 + 1) = 4000$ I/Os
- Compare to 2-Way: 22000 I/Os (5.5x improvement!)

5.3 Understanding Buffer Allocation: The Core Problem

The Fundamental Question: Why Does B Matter?

The Central Problem Tiantian Liu is Addressing:
When sorting data that doesn't fit in memory, we have **limited buffer pages (B)**. The key insight is: **how should we divide these B pages between different tasks?**
The Shift in Focus – From "What" to "How Much":

- **2-Way Sort:** We assumed minimal memory (3 pages) – simple but wasteful
- **Multi-Way Sort:** Now we ask: "We have B pages – how to use them optimally?"
- **Key Realization:** More memory = bigger initial runs + merging more runs at once

Why This Matters:

- **Real databases** have varying amounts of available memory
- **Same algorithm** behaves very differently with different B values
- **The optimizer** needs to know B to estimate sorting costs accurately

How B (Buffer Pages) is Determined

Where Does B Come From?
B is **NOT** a formula you calculate – it's a **given constraint** from the system!
Sources of B:

- **Database configuration:** DBA sets buffer pool size (e.g., 1GB)
- **Page size:** System defines page size (e.g., 8KB)
- **B = Buffer Pool / Page Size:** e.g., 1GB/8KB = $131,072$ pages
- **For sorting:** Only a fraction may be allocated (e.g., B = 1000 pages)

In Exam Problems:

- B is **always given** (e.g., "You have B = 100 buffer pages")
- Sometimes called M instead of B (same meaning!)
- **Your job:** Use B to calculate costs, not to find B

The Constraint:

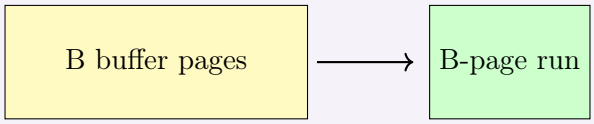
Everything must fit in B pages at any moment

Buffer Allocation Strategy: The Key Insight

The Problem: We have B buffer pages. How to divide them?
Phase 1 – Creating Runs:

- **Use ALL B pages** for sorting
- Read B pages → Sort in memory → Write B pages

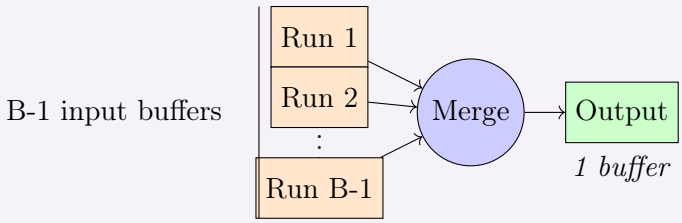
- Result: Each run is B pages long
- **Why use all B?** Longer runs = fewer runs = fewer merge passes later!



All B pages used for sorting

Phase 2 – Merging Runs:

- **Problem:** Need to merge multiple runs simultaneously
- **Constraint:** Need 1 input buffer per run being merged + 1 output buffer
- **Solution:** Can merge at most **(B-1) runs** at a time



Total: (B-1) + 1 = B buffers

The Trade-off Explained:

- **Why B-1?** We need 1 page for output – can't use all B for input
- **Why not less?** Using fewer input buffers = more passes = more I/O
- **Optimal:** Always use maximum (B-1) input buffers when possible

Worked Example: Buffer Allocation in Action

Given: N = 1200 pages to sort, B = 100 buffer pages
Phase 1 – How many runs?

- Each run uses all B = 100 pages
- Number of runs = $\lceil N/B \rceil = \lceil 1200/100 \rceil = 12$ runs
- Each run is 100 pages long (except last may be smaller)

Phase 2 – How many passes?

- We can merge (B-1) = 99 runs at a time
- We have 12 runs to merge
- Since $12 \leq 99$: **Only 1 pass needed!**
- All 12 runs fit in our merge capacity

Buffer Layout in Phase 2:

Purpose	Pages Used
Input buffer for Run 1	1 page
Input buffer for Run 2	1 page
⋮	⋮
Input buffer for Run 12	1 page
(Unused input buffers)	*87 pages*
Output buffer	1 page
Total	**100 pages = B**

Total Cost:

$$\text{Cost} = 2N \times (\text{passes} + 1) = 2 \times 1200 \times (1 + 1) = 4800 \text{ I/Os}$$

Common Confusion: What If Runs Don't Fit?

Question: What if we have MORE runs than (B-1)?
Example: N = 10,000 pages, B = 10 buffers

- Phase 1: $\lceil 10000/10 \rceil = 1000$ runs (each 10 pages)
- But we can only merge $(B - 1) = 9$ runs at a time!

Solution: Multiple Merge Passes

- **Pass 1:** Merge groups of 9 runs $\rightarrow \lceil 1000/9 \rceil = 112$ runs
- **Pass 2:** Merge groups of 9 $\rightarrow \lceil 112/9 \rceil = 13$ runs
- **Pass 3:** Merge groups of 9 $\rightarrow \lceil 13/9 \rceil = 2$ runs
- **Pass 4:** Merge 2 runs $\rightarrow$ 1 final sorted file

Number of Passes Formula:

$$\text{Passes} = \lceil \log_{B-1}(\text{initial runs}) \rceil = \lceil \log_9(1000) \rceil = 4$$

Key Insight: More buffer pages (larger B) = merge more runs at once = fewer passes needed!

Summary: The B-Centric View of External Sorting

Everything Revolves Around B:

Phase	How B is Used	Effect of Larger B
Phase 1	Create runs of B pages	Longer runs
Phase 2	Merge (B-1) runs at once	Fewer passes

The Two Key Formulas:

1. **Number of initial runs:** $\lceil N/B \rceil$
2. **Number of merge passes:** $\lceil \log_{B-1}(\lceil N/B \rceil) \rceil$

Why More Memory Helps Dramatically:

- Double B → Half as many initial runs
- Double B → Can merge twice as many runs per pass
- Combined effect: Logarithmic reduction in total passes!

Practical Implication:

- With B = 1000 pages, can sort $B \times (B - 1) = 999,000$ pages in just 2 passes!
- This is why databases allocate significant memory for sorting operations

6 Join Strategies

Why Joins Matter - The Big Picture

What is a Join?

- **Goal:** Combine two tables based on a common attribute
- **Example:** Students table + Enrollment table → Who took what course?
- **Plain English:** Match rows from table R with rows from table S

Why is Joining Hard?

- **Brute force:** Compare every row in R with every row in S
- **Problem:** If R has 1000 rows and S has 1000 rows = 1,000,000 comparisons!
- **Worse:** Tables might be too big to fit in memory
- **Challenge:** How to join efficiently using limited memory?

The Fundamental Trade-off:

- **Simple algorithms:** Easy to implement, but SLOW
- **Smart algorithms:** More complex, but MUCH faster
- **Key factors:** Memory available, whether data is sorted, join condition type

Real-World Impact:

- **Bad join:** Query takes hours, database grinds to halt
- **Good join:** Query returns in seconds, users happy
- **The difference:** Choosing the right algorithm!

Join Overview

Problem: Compute $R \bowtie S$ where R and S are relations
Plain English: Find all matching pairs of rows from tables R and S
Notation (Important for Cost Formulas):

- $|R|$ = number of pages in R

- **Plain English:** How many disk pages R occupies
- $|S|$ = number of pages in S
- **Plain English:** How many disk pages S occupies
- p_r = number of tuples (rows) per page in R
- **Plain English:** How many rows fit in one page of R
- p_s = number of tuples (rows) per page in S
- **Plain English:** How many rows fit in one page of S
- B = number of buffer pages available
- **Plain English:** How much memory we have for the join operation

Strategies Covered (From Simple to Sophisticated):

1. Simple Nested-Loop Join - **Easiest but slowest**
2. Page-Oriented Nested-Loop Join - **Better than simple**
3. Block Nested-Loop Join - **Uses available memory wisely**
4. Sort-Merge Join - **Good when data is sorted**
5. Hash Join - **Usually the fastest!**

Key Principle for All Join Algorithms:

- **Critical rule:** Always use the SMALLER table as the outer relation
- **Why?** Outer table is scanned once; inner table scanned many times
- **Plain English:** The table you scan repeatedly should be as small as possible!

Join Strategy Selection Guidelines

Use Simple/Page/Block Nested-Loop when:

- No indexes available
- Inner relation is small
- Buffer space is available (for Block NL)

Use Sort-Merge Join when:

- One or both inputs already sorted
- Output required to be sorted
- Large relations with sufficient memory

Use Hash Join when:

- Equi-joins (equality condition)

- Relations not sorted
- Good hash distribution expected

6.1 Nested-Loop Join Variants

Understanding Nested-Loop Joins

The Core Idea:

- **Concept:** Like nested loops in programming!
- Outer loop: Go through R
- Inner loop: For each R row, search all of S for matches
- **Analogy:** Like checking every person against every other person at a party

Three Variants - What's the Difference?

- **Simple:** Loop through individual ROWS (tuples)
- **Page-Oriented:** Loop through PAGES instead
- **Block:** Load MULTIPLE pages of outer table at once
- **Key progression:** Each variant is smarter about disk I/O!

Simple Nested-Loop Join

The Most Basic Approach:
Algorithm (Pseudocode):

```
foreach tuple r in R do          // For each ROW in R
    foreach tuple s in S do      // For each ROW in S
        if r.ID == s.ID then add <r, s> to result
```

Plain English:

- Take first row from R
- Compare it with EVERY row in S
- Repeat for second row from R, third row, etc.
- **Problem:** For every single row in R, we scan ALL of S!

I/O Cost (R as outer):

- Scan R once: $|R|$ I/Os
- **Plain English:** Read R completely
- For each tuple in R, scan S: $(p_r \times |R|) \times |S|$ I/Os
- **Plain English:** R has $(p_r \times |R|)$ total tuples, for each we scan all $|S|$ pages of S
- **Total:** $|R| + (p_r \times |R|) \times |S|$

Example: $|R| = 1000$ pages, $|S| = 500$ pages, $p_r = 100$ tuples/page

- R has $100 \times 1000 = 100,000$ tuples
- For each tuple, scan 500 pages
- Cost $= 1000 + (100 \times 1000) \times 500 = 1000 + 50,000,000 = 50,001,000$ I/Os
- **Plain English:** FIFTY MILLION disk accesses! Extremely slow!

Key Insight:

- **Very expensive!** Scans inner relation for every tuple
- **Why so bad?** We re-read the same pages of S over and over
- **Critical:** Use smaller relation as outer (reduces number of S scans)
- **Lesson:** This algorithm is terrible - only use as last resort!

Page-Oriented Nested-Loop Join

Improvement: Scan inner relation once per page (not per tuple)
Algorithm:

```
foreach page pr in R do
    foreach page ps in S do
        foreach tuple r in pr do
            foreach tuple s in ps do
                if r.ID == s.ID then add <r, s> to result
```

I/O Cost (R as outer):

- Scan R once: $|R|$ I/Os
- For each page in R, scan S: $|R| \times |S|$ I/Os
- **Total:** $|R| + |R| \times |S|$

Example: $|R| = 1000$, $|S| = 500$

- Cost $= 1000 + 1000 \times 500 = 501,000$ I/Os
- 100x better than simple nested-loop!

Block Nested-Loop Join

Improvement: Use (B-2) pages for outer relation
Buffer Allocation:

- (B-2) pages for outer relation block
- 1 page for inner relation input
- 1 page for output

Algorithm:

- Read (B-2) pages of R into memory
- Scan entire S, joining with all tuples in R block
- Repeat for next block of R

I/O Cost (R as outer):

- Scan R once: $|R|$ I/Os
- Number of R blocks: $\lceil |R|/(B-2) \rceil$
- Per R block, scan S: $\lceil |R|/(B-2) \rceil \times |S|$ I/Os
- **Total:** $|R| + \lceil |R|/(B-2) \rceil \times |S|$

Example: $|R| = 1000$, $|S| = 500$, $B = 102$

- R blocks: $\lceil 1000/100 \rceil = 10$
- Cost $= 1000 + 10 \times 500 = 6000$ I/Os
- 83x better than page-oriented!

Key Insight:

- Use smaller relation as outer
- More buffer pages $\rightarrow$ fewer I/Os

6.2 Sort-Merge Join

Sort-Merge Join Algorithm

Steps:

1. Sort R on join attribute
2. Sort S on join attribute
3. Scan sorted R and sorted S in tandem to find matches

Merge Phase:

- Maintain pointers to current positions in R and S
- Advance pointer for relation with smaller current value
- When values match, output all matching pairs

I/O Cost:

- Sort R: $2|R| \times (\lceil \log_{B-1} \lceil |R|/B \rceil \rceil + 1)$
- Sort S: $2|S| \times (\lceil \log_{B-1} \lceil |S|/B \rceil \rceil + 1)$
- Merge: $|R| + |S|$
- **Total: Sort R + Sort S** $+ |R| + |S|$

Refinement:

- If enough memory: $|R|/B + |S|/B < B$
- Can combine merge with last sort pass
- **Optimized Cost:** $3(|R| + |S|)$

When to Use:

- One or both inputs already sorted
- Output required to be sorted
- Large relations (amortize sorting cost)

6.3 Hash Join

Hash Join Algorithm

Key Idea: Partition both relations using hash function
Property:

- R tuples in partition i will only match S tuples in partition i
- $R \bowtie S = (R_1 \bowtie S_1) \cup (R_2 \bowtie S_2) \cup ... \cup (R_{B-1} \bowtie S_{B-1})$

Phase 1: Partitioning

1. Hash each tuple of R using hash function h
2. Place tuple in bucket h(join attribute)
3. Repeat for S
4. Uses B buffer pages (B-1 output partitions + 1 input)

Phase 2: Matching

1. For each partition $i = 1, 2, ..., B - 1$:
2. Load R_i into memory (build hash table)
3. Scan S_i, probe hash table for matches

I/O Cost:

- **Partitioning Phase**: Read + write both relations = $2(|R| + |S|)$
- **Matching Phase**: Read both relations = $|R| + |S|$
- **Total:** $3(|R| + |S|)$

When to Use:

- Equi-joins (equality condition on join attribute)
- Relations not already sorted
- Uniform hash distribution

7 Query Optimization

Why Query Optimization Matters

The Big Challenge:

- **Problem:** Many ways to execute the same query
- **Example:** Join A, B, C - could do $(A \bowtie B) \bowtie C$ or $A \bowtie (B \bowtie C)$
- **Reality:** Different orders = vastly different performance!
- **Impact:** Wrong order = query takes 1 hour; right order = 1 second

The Query Optimizer's Job:

- **Input:** SQL query (declarative - says WHAT, not HOW)

- **Output:** Execution plan (says exactly HOW to execute)
- **Goal:** Find the cheapest (fastest) execution plan
- **Challenge:** Explore many options without taking forever

Two Types of Optimization:

- **Logical Optimization:** Apply algebraic rules (always beneficial)
- **Plain English:** Rewrite query to be more efficient (e.g., push filters down)
- **Physical Optimization:** Choose algorithms and access paths
- **Plain English:** Decide which join algorithm, which index to use

Golden Rule of Query Optimization:

- **Reduce data as early as possible!**
- Apply filters (WHERE clauses) before joins
- Remove unneeded columns (projections) early
- **Why?** Smaller data = faster operations downstream

7.1 Cost Estimation

Statistical Information

Catalog Statistics:

- n_r = number of tuples in relation r
- b_r = number of blocks containing tuples of r
- f_r = blocking factor (tuples per block)
- $V(A, r)$ = number of distinct values for attribute A in r

Relationship:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil \tag{3}$$

Selection Size Estimation

Condition: $\sigma_{A=v}(r)$

- Estimated result size: $c = n_r/V(A, r)$
- **Assumption**: Values uniformly distributed

Condition: $\sigma_{A\leq v}(r)$

- If $\min(A, r)$ and $\max(A, r)$ available:
- $c = 0$ if $v < \min(A, r)$
- $c = n_r$ if $v \geq \max(A, r)$

- $c = n_r \times \frac{v - \min(A,r)}{\max(A,r) - \min(A,r)}$ otherwise

Selection Size Estimation Example

Given: Relation $r(A, B, C)$ with $n_r = 5000$, $V(B, r) = 100$
Q1: Estimate size of $\sigma_{B=3}(r)$

- $c = n_r/V(B, r) = 5000/100 = 50$ tuples

Q2: Range of A is [10, 60], uniformly distributed. Estimate size of $\sigma_{A<20}(r)$

- $c = n_r \times \frac{20-10}{60-10} = 5000 \times \frac{10}{50} = 1000$ tuples

7.2 Logical Query Optimization

Equivalence Rules

Rule 1: Conjunctive Selection

- $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$

Rule 2: Selection Commutativity

- $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$

Rule 3: Projection Cascade

- $\pi_{L_1}(\pi_{L_2}(...(\pi_{L_n}(E))...)) \equiv \pi_{L_1}(E)$ where $L_1 \subseteq L_2 \subseteq ... \subseteq L_n$

Rule 4: Combine Selection with Cartesian Product

- $\sigma_\theta(E_1 \times E_2) \equiv E_1 \bowtie_\theta E_2$

Rule 5: Join Commutativity

- $E_1 \bowtie_\theta E_2 \equiv E_2 \bowtie_\theta E_1$

Logical Optimization Steps

Step 1: Break up and push down selections

- Decompose conjunctive selections
- Push selections as close to leaves as possible
- **Benefit**: Reduce intermediate result sizes

Step 2: Introduce joins

- Combine selections with Cartesian products
- Replace $\sigma_\theta(R \times S)$ with $R \bowtie_\theta S$

Step 3: Determine join order

- **Heuristic**: Execute joins with selection input first

- Smaller intermediate results → lower cost

Step 4: Introduce and push down projections

- Remove attributes not needed for final result
- Push projections down to reduce tuple size

Logical Optimization Example

Given: Relations $r(A, B)$, $s(C, D)$
Query: $\pi_A(\sigma_{A=5 \wedge B<D}(r \times s))$
Step 1: Break up and push down selections

- $\pi_A(\sigma_{B<D}(\sigma_{A=5}(r) \times s))$
- Push $\sigma_{A=5}$ to r: $\pi_A(\sigma_{B<D}(\sigma_{A=5}(r) \times s))$

Step 2: Introduce join

- $\pi_A(\sigma_{A=5}(r) \bowtie_{B<D} s)$

Step 3: Push down projection

- $\pi_A(\pi_{A,B}(\sigma_{A=5}(r)) \bowtie_{B<D} \pi_D(s))$

Final Optimized Query:

- $\pi_A(\pi_{A,B}(\sigma_{A=5}(r)) \bowtie_{B<D} \pi_D(s))$

8 Summary - Cost Formulas

Quick Reference - I/O Costs

Sorting:

- 2-Way External Merge Sort: $2N \times (\lceil \log_2 N \rceil + 1)$
- Multi-Way External Merge Sort: $2N \times (\lceil \log_{B-1} \lceil N/B \rceil \rceil + 1)$

Join (R outer, S inner):

- Simple Nested-Loop: $|R| + (p_r \times |R|) \times |S|$
- Page-Oriented Nested-Loop: $|R| + |R| \times |S|$
- Block Nested-Loop: $|R| + \lceil |R|/(B-2) \rceil \times |S|$
- Sort-Merge: Sort R + Sort S + $|R| + |S|$ (or $3(|R| + |S|)$ if optimized)
- Hash Join: $3(|R| + |S|)$

Key Principles:

- Use smaller relation as outer in nested-loop joins
- More buffer pages $\rightarrow$ fewer I/Os
- Push selections down to reduce intermediate results

Join Cost Comparison Table

Join Type	I/O Cost	Buffer Pages Used
Simple NL	$\lvert R\rvert + (p_r \cdot \lvert R\rvert) \cdot \lvert S\rvert$	2
Page NL	$\lvert R\rvert + \lvert R\rvert \cdot \lvert S\rvert$	3
Block NL	$\lvert R\rvert + \lceil \lvert R\rvert/(B-2) \rceil \cdot \lvert S\rvert$	B
Sort-Merge	$3(\lvert R\rvert + \lvert S\rvert)$ (optimized)	B
Hash Join	$3(\lvert R\rvert + \lvert S\rvert)$	B

9 Practice Exercises

9.1 Exercise 1: External Merge Sort

Practice - Sorting Cost Calculation

Problem: Relation r has 40,000 tuples, 40 tuples per page. Assume B pages of memory and $B \leq 500$.

Tasks:

1. Calculate number of pages N
2. Estimate I/O cost for Multi-Way External Merge Sort
3. How many passes in Phase 2?
4. Compare with 2-Way External Merge Sort

Solution:

1. Number of pages:

- $N = 40,000/40 = 1000$ pages

2. **Multi-Way cost (assume $B = 100$):**
   - Number of runs after Phase 1: $\lceil N/B \rceil = \lceil 1000/100 \rceil = 10$
   - Passes in Phase 2: $\lceil \log_{B-1} \lceil N/B \rceil \rceil = \lceil \log_{99} 10 \rceil = 1$
   - Cost $= 2N \times (\text{passes} + 1) = 2 \times 1000 \times (1 + 1) = 4000$ I/Os
3. **Passes in Phase 2:** 1 pass
4. **Compare with 2-Way:**
   - 2-Way passes: $\lceil \log_2 1000 \rceil = 10$
   - 2-Way cost: $2 \times 1000 \times (10 + 1) = 22,000$ I/Os
   - Multi-Way is 5.5x better!

9.2 Exercise 2: Join Cost Calculation

Practice - Nested-Loop Join Costs

Problem: Relations r_1 and r_2:

- r_1: 40,000 tuples, 40 tuples per page
- r_2: 50,000 tuples, 25 tuples per page
- $B = 100 + 2 = 102$ buffer pages

Tasks: Calculate I/O cost for:

1. Simple nested-loop join (r_1 outer)
2. Page-oriented nested-loop join (r_1 outer)
3. Block nested-loop join (r_1 outer)
4. Which relation should be outer? Why?

Solution:
Setup:

- $N_1 = 40,000/40 = 1000$ pages, $p_1 = 40$
- $N_2 = 50,000/25 = 2000$ pages, $p_2 = 25$

1. **Simple nested-loop (r_1 outer):**
   - Cost $= |N_1| + p_1 \times |N_1| \times |N_2|$
   - Cost $= 1000 + 40 \times 1000 \times 2000 = 80,001,000$ I/Os
2. **Page-oriented nested-loop (r_1 outer):**
   - Cost $= |N_1| + |N_1| \times |N_2|$
   - Cost $= 1000 + 1000 \times 2000 = 2,001,000$ I/Os
3. **Block nested-loop (r_1 outer):**

- Cost $= |N_1| + \lceil |N_1|/(B-2) \rceil \times |N_2|$
- Cost $= 1000 + \lceil 1000/100 \rceil \times 2000 = 1000 + 10 \times 2000 = 21,000$ I/Os

4. Which relation should be outer?

- r_1 outer: 21,000 I/Os (calculated above)
- r_2 outer: $2000 + \lceil 2000/100 \rceil \times 1000 = 2000 + 20 \times 1000 = 22,000$ I/Os
- **Answer: r_1 should be outer (smaller relation)**

9.3 Exercise 3: Sort-Merge and Hash Join

Practice - Sort-Merge and Hash Join Costs

Problem: Same relations as Exercise 2:

- $N_1 = 1000$ pages, $N_2 = 2000$ pages
- $B \leq 500$ buffer pages

Tasks:

1. Calculate cost for sort-merge join
2. Calculate cost for hash join
3. Compare with best nested-loop join

Solution:
1. Sort-Merge Join:
Check if optimized formula applies: $N_1/B + N_2/B < B$

- $1000/B + 2000/B = 3000/B$
- If $B = 100$: $3000/100 = 30 < 100$ ✓
- **Can use optimized formula**

Cost $= 3(N_1 + N_2) = 3 \times (1000 + 2000) = 9000$ I/Os
2. Hash Join:

- Cost $= 3(N_1 + N_2) = 3 \times 3000 = 9000$ I/Os

3. Comparison:

- Best nested-loop (Block): 21,000 I/Os
- Sort-Merge: 9,000 I/Os (2.3x better)
- Hash Join: 9,000 I/Os (2.3x better)
- **Sort-Merge and Hash Join are equally efficient**

9.4 Exercise 4: Logical Query Optimization

Practice - Query Optimization

Problem: Optimize the following query using logical optimization rules.
Relations:

- $r(A, B)$
- $s(C, D)$

Query: $\pi_A(\sigma_{A=5 \wedge B<D}(r \times s))$
Tasks:

1. Draw initial query tree
2. Apply optimization rules
3. Draw final optimized query tree
4. Explain why the optimized version is better

Solution:
1. Initial Query Tree:

```
        Pi_A
         |
    sigma_(A=5 AND B<D)
         |
      r x s
     /     \
    r       s
```

2. Apply Optimization Rules:
Step 1: Break up conjunctive selection

```
        Pi_A
         |
      sigma_(B<D)
         |
      sigma_(A=5)
         |
      r x s
```

Step 2: Push down $\sigma_{A=5}$ (only references r)

```
        Pi_A
         |
      sigma_(B<D)
         |
      r x s
     /     \
  sigma_(A=5)  s
    |
    r
```

Step 3: Combine selection with Cartesian product (introduce join)

```
        Pi_A
         |
    r |X|_(B<D) s
     /     \
  sigma_(A=5)  s
    |
    r
```

Step 4: Push down projections

```
        Pi_A
         |
    Pi_(A,B) |X|_(B<D) Pi_D
     /           \
  sigma_(A=5)    s
    |
    r
```

3. Final Optimized Query:

$$\pi_A(\pi_{A,B}(\sigma_{A=5}(r)) \bowtie_{B<D} \pi_D(s))$$

4. Why is this better?

- $\sigma_{A=5}$ applied early: Reduces size of r before join
- Cartesian product replaced with join: More efficient
- Projections reduce tuple size: Smaller intermediate results
- Overall: Much smaller intermediate results $\rightarrow$ lower I/O cost

9.5 Exercise 5: Join Strategy Selection

Practice - Choosing Join Strategy

Scenario: You need to join two relations:

- R: 10,000 pages, unsorted
- S: 5,000 pages, sorted on join attribute
- $B = 200$ buffer pages

Tasks:

1. Which join strategy would you choose? Why?
2. Calculate the cost of your chosen strategy
3. What if R were also sorted on the join attribute?
4. What if $B = 50$ instead?

Solution:
1. Strategy Selection:
Option A: Block Nested-Loop

- Cost $= 5000 + \lceil 5000/198 \rceil \times 10000 \approx 5000 + 26 \times 10000 = 265,000$
- (Use S as outer since it's smaller)

Option B: Sort-Merge

- Need to sort R: $2 \times 10000 \times (\lceil \log_{199} \lceil 10000/200 \rceil \rceil + 1)$
- $\lceil 10000/200 \rceil = 50$ runs
- $\lceil \log_{199} 50 \rceil = 1$ pass
- Sort cost: $2 \times 10000 \times 2 = 40,000$
- Merge cost: $10000 + 5000 = 15,000$
- Total: $40,000 + 15,000 = 55,000$ I/Os

Option C: Hash Join

- Cost $= 3(10000 + 5000) = 45,000$ I/Os

Best Choice: Hash Join (45,000 I/Os)
2. Cost: 45,000 I/Os
3. If R also sorted:

- Sort-Merge would be best (no sorting needed!)
- Cost = merge only $= 10000 + 5000 = 15,000$ I/Os

4. If $B = 50$:

- Block nested-loop: $5000 + \lceil 5000/48 \rceil \times 10000 \approx 5000 + 105 \times 10000 = 1,055,000$
- Hash join: Still $45,000$ (independent of B for basic version)
- Hash join remains best

9.6 Exercise 6: Comprehensive Problem

Practice - End-to-End Query Optimization

Problem: Consider the following query on relations:

- `instructor`$(ID, name, dept_name, salary)$
- `teaches`$(ID, course_id, semester, year)$

Query:

```
SELECT t.course_id
FROM instructor i, teaches t
WHERE i.name = 'Brandt' AND t.ID = i.ID;
```

Statistics:

- instructor: 1000 pages, 100 tuples/page
- teaches: 2000 pages, 50 tuples/page

- $V(\text{name}, \text{instructor}) = 5000$ (all unique)
- $B = 102$ buffer pages

Tasks:

1. Write the initial relational algebra expression
2. Optimize using logical optimization rules
3. Estimate size of $\sigma_{\text{name}='Brandt'}(\texttt{instructor})$
4. Choose the best join strategy
5. Calculate total query cost

Solution:
1. Initial Expression:

$$\pi_{\text{course_id}}(\sigma_{\text{name}='Brandt' \wedge \text{t.ID}=\text{i.ID}}(\texttt{instructor} \times \texttt{teaches}))$$

2. Optimized Expression:
Step-by-step optimization:

- Break up selection: $\sigma_{\text{t.ID}=\text{i.ID}}(\sigma_{\text{name}='Brandt'}(\texttt{instructor}) \times \texttt{teaches})$
- Push down selection: $\sigma_{\text{t.ID}=\text{i.ID}}(\sigma_{\text{name}='Brandt'}(\texttt{instructor}) \times \texttt{teaches})$
- Introduce join: $(\sigma_{\text{name}='Brandt'}(\texttt{instructor})) \bowtie_{\text{ID}} \texttt{teaches}$
- Push projection: $\pi_{\text{course_id}}(\pi_{\text{ID}}(\sigma_{\text{name}='Brandt'}(\texttt{instructor})) \bowtie_{\text{ID}} \pi_{\text{ID, course_id}}(\texttt{teaches}))$

3. Size Estimation:

- $|\sigma_{\text{name}='Brandt'}(\texttt{instructor})| = 100,000/5000 = 20$ tuples
- Assuming 100 tuples/page: ≈ 1 page

4. Join Strategy:
After selection, we're joining:

- Filtered instructor: ≈ 1 page
- teaches: 2000 pages

Block Nested-Loop (filtered instructor as outer):

- Cost $= 1 + \lceil 1/100 \rceil \times 2000 = 1 + 1 \times 2000 = 2001$ I/Os

5. Total Query Cost:

- Selection scan: 1000 I/Os
- Join: 2001 I/Os
- **Total: 3001 I/Os**

Key Insight:

- Selection reduces instructor from 1000 pages to 1 page
- Makes join very efficient!
- This is why pushing selections down is crucial

10 Advanced Topics

10.1 Query Processing Pipeline

Query Processing Flow

Complete Pipeline:

1. **Parsing**: SQL → Parse Tree
2. **Translation**: Parse Tree → Relational Algebra
3. **Logical Optimization**: Apply equivalence rules
4. **Physical Planning**: Choose algorithms and access paths
5. **Execution**: Run the plan and return results

Key Components:

- Parser: Validates syntax, builds parse tree
- Translator: Converts to relational algebra
- Optimizer: Uses statistics and rules to find efficient plan
- Execution Engine: Implements physical operators

10.2 Additional Join Examples

Practice - Comparing Join Strategies

Problem: r_1 has 20,000 tuples (800 pages, 25 tuples/page)
r_2 has 45,000 tuples (1500 pages, 30 tuples/page)
$B = 102$ buffer pages
Calculate costs for $r_1 \bowtie r_2$:
Simple Nested-Loop (r_1 outer):

- Cost $= 800 + (20,000 \times 1500) = 30,000,800$ I/Os

Page-Oriented Nested-Loop (r_1 outer):

- Cost $= 800 + (800 \times 1500) = 1,200,800$ I/Os

Block Nested-Loop (r_1 outer):

- Cost $= 800 + \lceil 800/100 \rceil \times 1500 = 800 + 8 \times 1500 = 12,800$ I/Os

Sort-Merge (if $1500/M + 800/M < M$):

- Cost $= 3(800 + 1500) = 6,900$ I/Os

Hash Join:

- Cost $= 3(800 + 1500) = 6,900$ I/Os

Winner: Hash Join or Sort-Merge (6,900 I/Os)

11 Tips for Success

Common Mistakes to Avoid

Sorting:

- Don't forget the "+1" in cost formula (includes Phase 1)
- Multi-Way uses (B-1) for merging, not B
- Remember: Each pass reads + writes all pages (factor of 2)

Joins:

- Always use smaller relation as outer (when applicable)
- Block nested-loop: $(B - 2)$ pages for outer, not $(B - 1)$
- Don't forget to scan outer relation once: $|R| + ...$

Query Optimization:

- Push selections down BEFORE introducing joins
- Cartesian product + selection = join
- Estimate intermediate result sizes for better join ordering

Study Strategy

For Exams:

1. Memorize cost formulas (or know how to derive them)
2. Practice calculating costs with different parameter values
3. Understand when each join strategy is best
4. Practice logical optimization step-by-step

Key Concepts to Master:

- External merge sort phases and cost
- All five join strategies and their costs
- Selection size estimation formulas
- Logical optimization rules and ordering

Practice Makes Perfect:

- Work through all exercises multiple times
- Try variations with different parameter values
- Compare costs of different strategies
- Understand trade-offs (buffer size, relation size, sorting)